Poutine Routine

A secure to-do application with a poutine collection

Course	**MOD8 - Advanced Web Programming (CEWP MOD8 GRB, Fall 2026)**
Institution	Concordia University, Continuing Education
Instructor	Marcos Sanches
Milestone	2 - Frontend Development
Team	Eliot Cupillard, Abigaïl Mialaret, Romain Denoual, Tiffany Vongchanh
Date	19 October 2026
Repository	https://github.com/Tezay/poutine-routine
Live demo	https://tezay.github.io/poutine-routine/

Contents

Frontend progress 3
- Summary 3
- Links and how to run it 3
- Key figures 3
- What works and what is test data 4

Screens 4
- Authentication 5
- Tasks 6
- Collection 10
- States and settings 11
- Differences from the Milestone 1 mockups 11

Client architecture 12
- Stack as built 12
- Folders 12
- How the pieces fit 13
- Frontend decisions 14

Accessibility 15

Project plan 15
- Planned vs Delivered 15
- Timeline 17
- Risks 17
- How we work 18

Challenges and solutions 18
- Merge conflicts on the router 18
- Test data must look like the API 18
- Dates in local time and UTC 18
- Highlight that blinked while dragging 18
- Images and Vite 18

Task distribution 19
- Milestone 2 - who did what 20

Tools 21

Frontend progress

Summary

Milestone 2 is the frontend. It is done for this milestone: six screens, ten components and one shared store, written in Vue 3 with Tailwind CSS. The application runs in the browser on test data. There is no back end yet. The back end, the database and the secure login come in Milestone 3.

The test data has the same shape as the future API: the field names are the database column names. In Milestone 3 we change where the data comes from. The screens stay the same.

The Milestone 1 document explains the problem, the solution, the database and the security plan. This document does not repeat them. It shows what we built, how it is organised, and what changes in the plan.

Links and how to run it

Item	Link
Repository	https://github.com/Tezay/poutine-routine (frontend in `client/`)
Live demo	https://tezay.github.io/poutine-routine/ (frontend only, test data)

To run the frontend on your computer (Node.js 20 or later):

```
git clone https://github.com/Tezay/poutine-routine.git
cd poutine-routine/client
npm install
cp .env.example .env
npm run dev
```

Then open `http://localhost:5173`.

Key figures

Figure	Value
Pull requests merged (19 October)	24
Feature issues done	15 of 17. The other two move to Milestone 3
Screens (one per route)	6
Components	10
Shared stores	1
Poutine sprites	26
Lines of Vue and JavaScript	about 1,700

What works and what is test data

The board is a real working frontend, but nothing is saved. A page reload brings back the test data.

Feature	State	Note
Board with three columns	Works	Reads the shared store
Create, edit and delete a task	Works	Title, description, status, due date, priority, tags
Drag and drop between columns	Works	The Status field of the task form is the keyboard way
Tag filter and search	Works	Several tags at once. Search looks at titles and tags
Table view	Works	Sort by clicking a column title
Countdown and overdue alert	Works	Updates every minute
Poutine collection	Works	Owned and locked poutines come from the test data
Box opening	Draw only	A random poutine appears. It costs nothing and is not saved
Companion	Display only	Shows the chosen poutine. The choice in Settings has no effect yet
Fries, daily goal and streak	Display only	Values come from the test data. The server will compute them
Register and login	Forms only	Fields are checked. There is no API call and no session
Settings	Form only	No change is saved

Everything marked “Display only” or “Forms only” needs the back end. It is planned for Milestone 3.

Screens

The images in this chapter are screenshots of the real application. They replace the mockups of Milestone 1. Each screen keeps the number of its mockup (01 to 10). Screens 11 to 14 are new.

The look follows the mockups: a left sidebar, a wide content area, and a right column for the companion. The colours and the fonts are the same.

Authentication

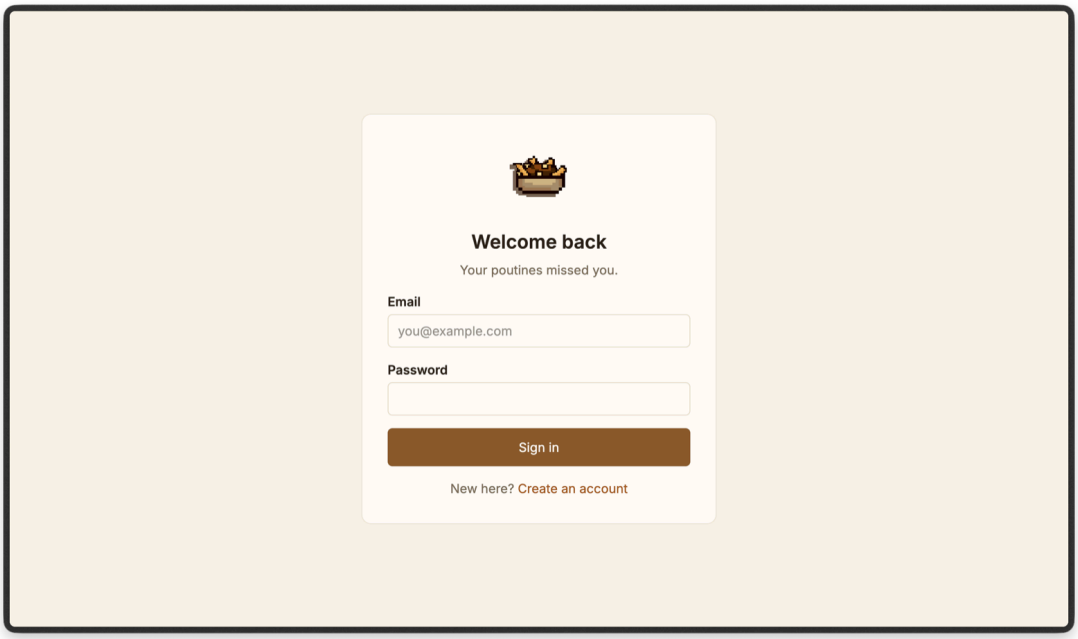


Figure 1: 01 - Login. A random poutine is the mascot.

The user enters an email and a password. Empty fields show a message. The form does not call an API yet. Login and register use the whole screen, without the sidebar.

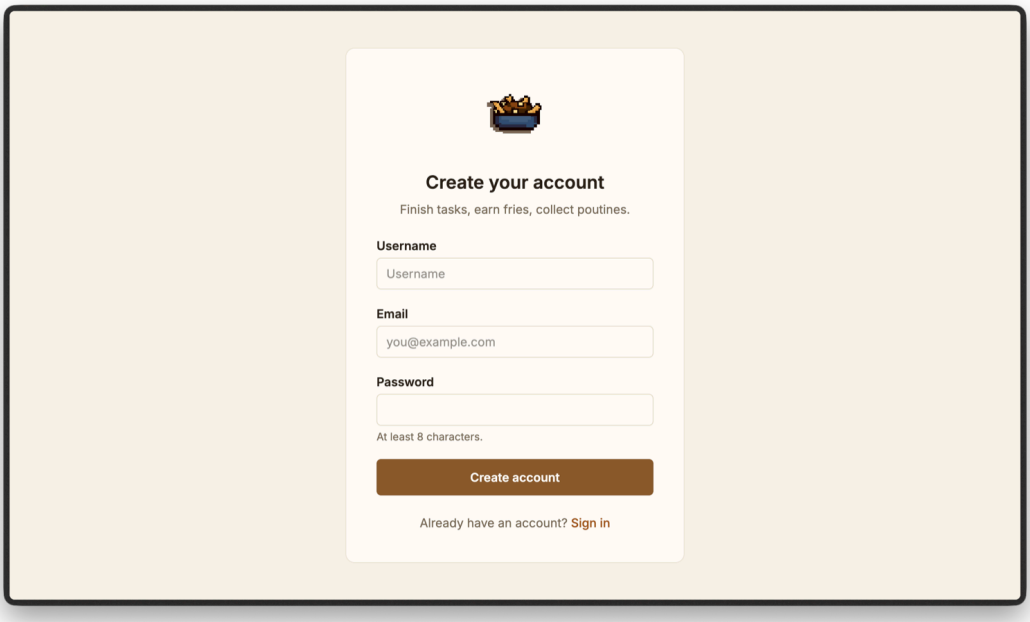


Figure 2: 02 - Register. Username, email and a password of at least 8 characters.

Tasks

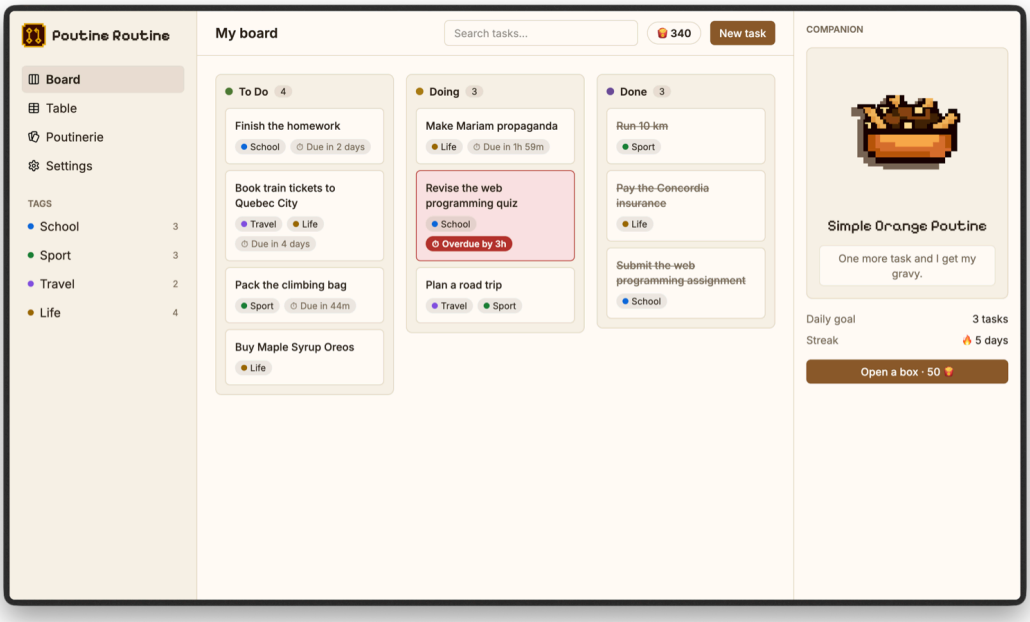


Figure 3: 03 - Board. Three columns, a card for each task, the companion on the right.

The board has three columns: To Do, Doing and Done. A card shows the title, the tags and the time left. Clicking a card opens the task form. Dragging a card to another column changes its status. Only the middle part of the page scrolls.

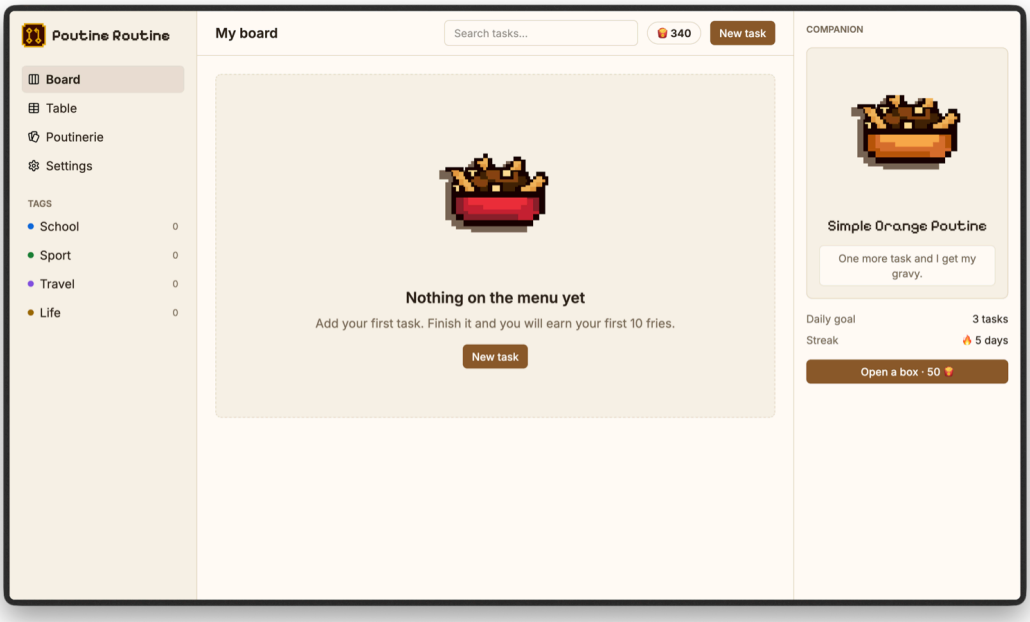


Figure 4: 04 - Empty board. The first screen of a new account.

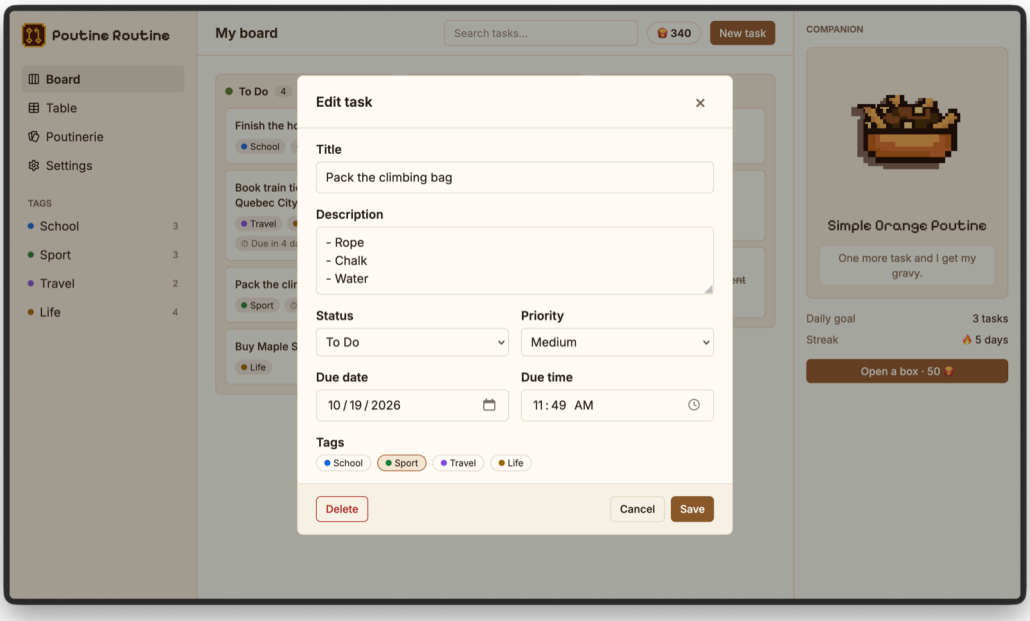


Figure 5: 05 - Task form, editing a task. It also has a Delete button.

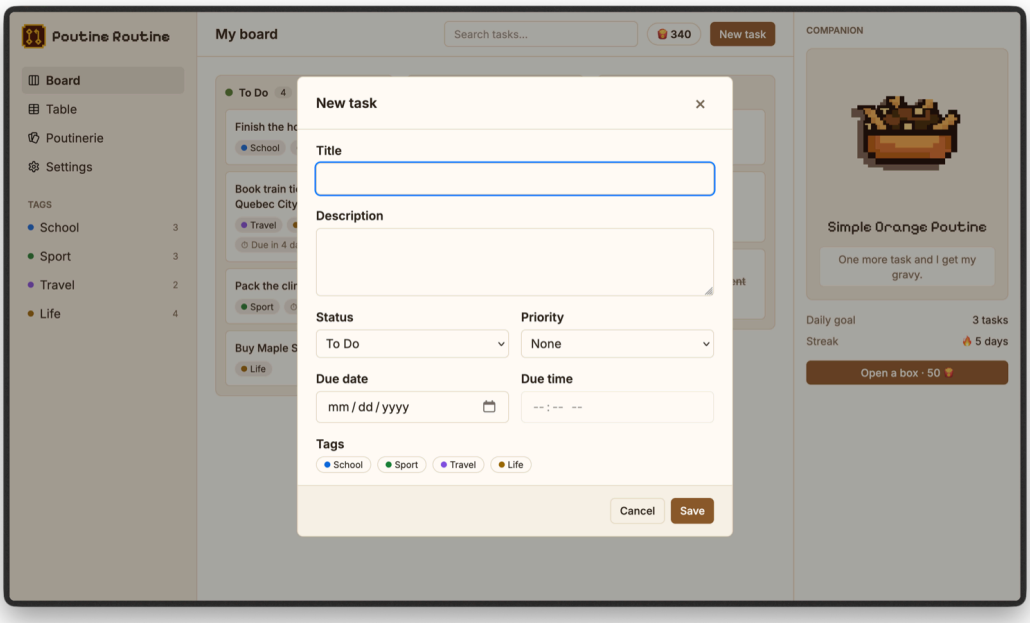


Figure 6: 11 - Task form, creating a task. Only the title is required.

The same form creates and edits a task. A due date without a time means the end of that day. Escape and a click outside close the form, and the focus returns to the button that opened it.

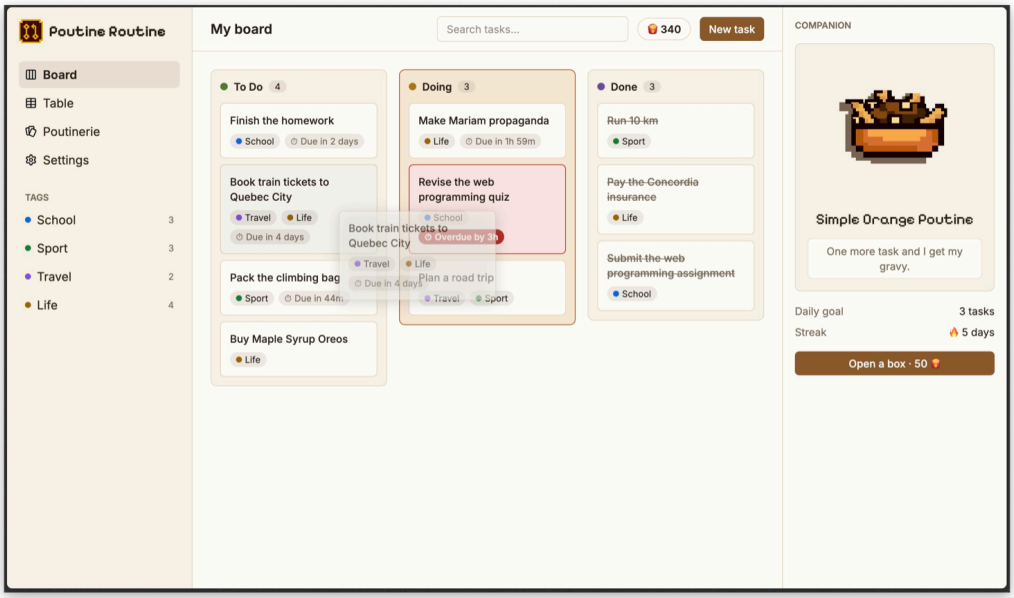


Figure 7: 13 - Drag and drop. The column under the card is highlighted.

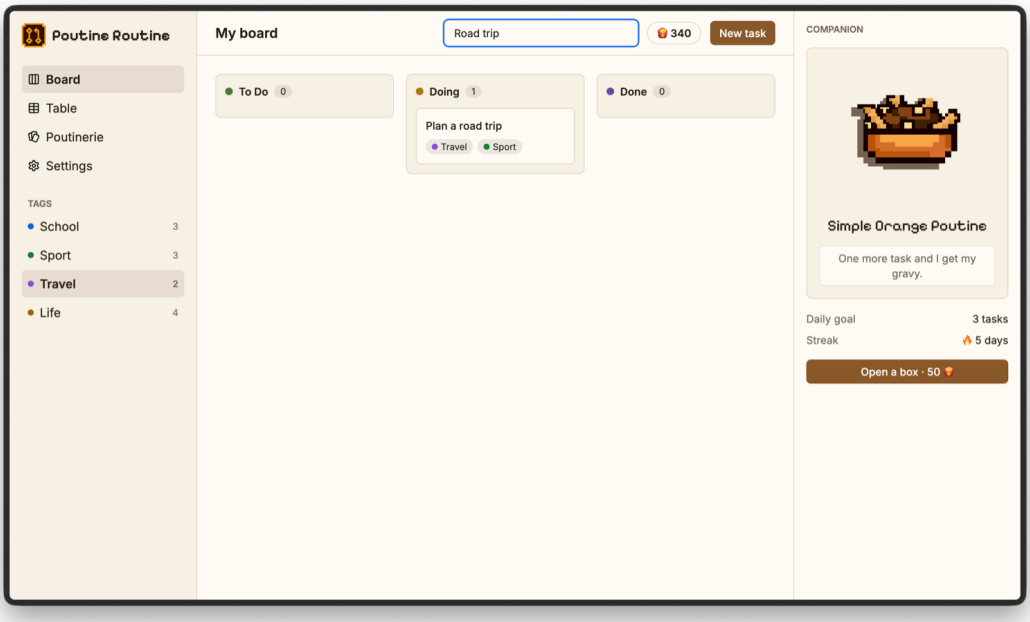


Figure 8: 12 - Search and tag filter. Clicking a tag turns it on or off.

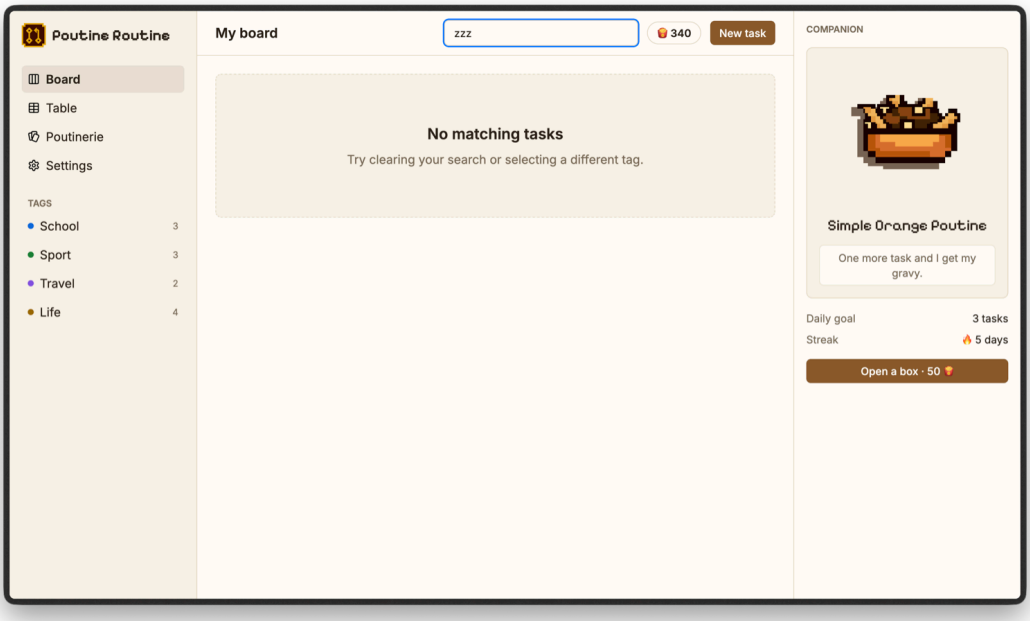


Figure 9: 14 - No result. A message appears when nothing matches.

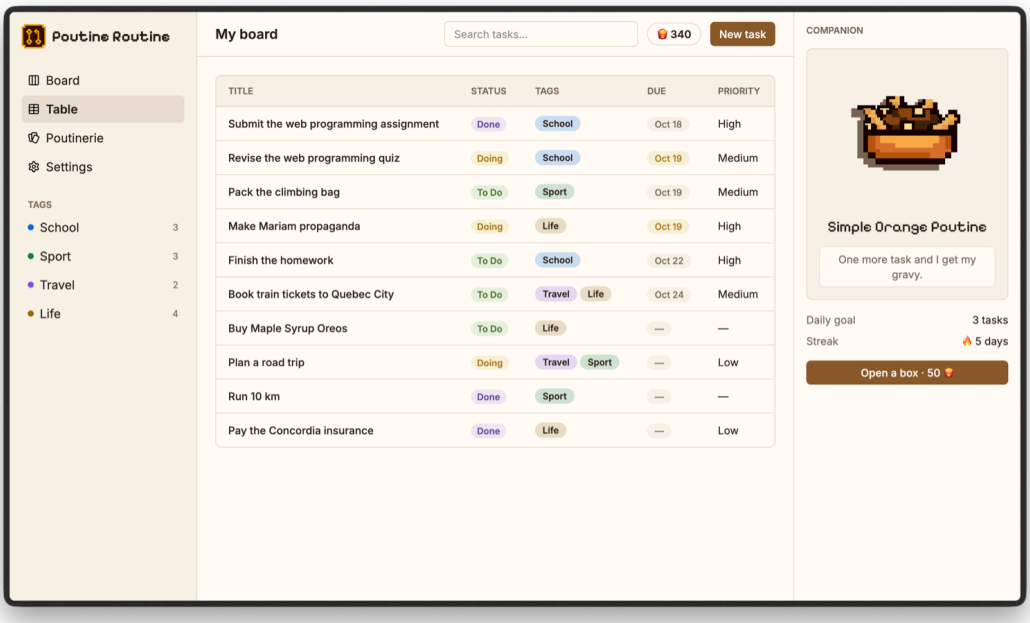


Figure 10: 06 - Table view. Click a column title to sort.

The table shows the same tasks as the board, with the same search and tag filter. A new task created on the board appears in the table at once, because both read the same store.

Collection

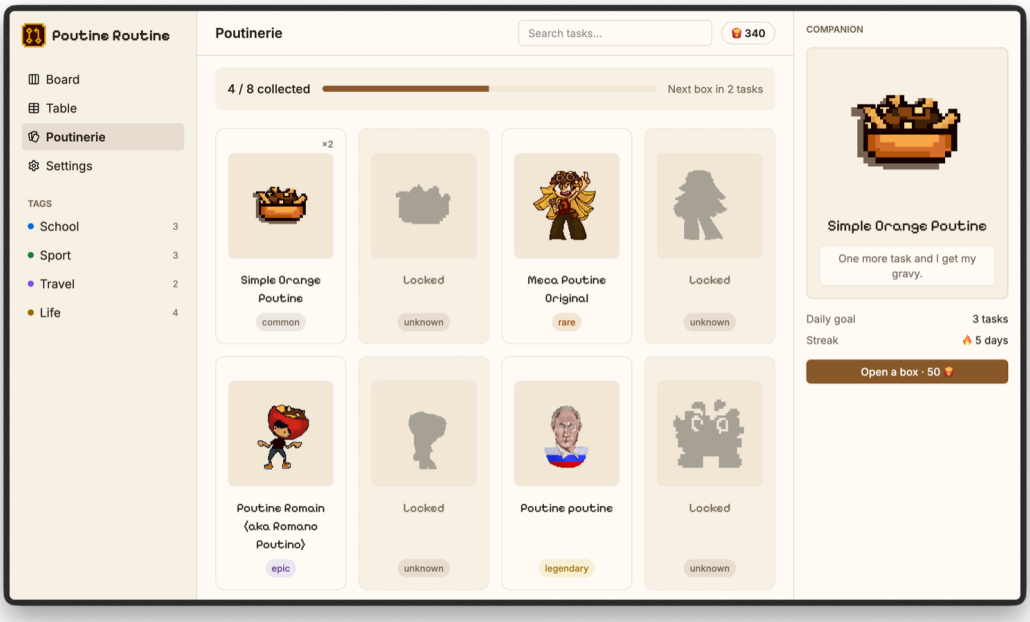


Figure 11: 07 - Poutinerie. Owned poutines with their rarity. Locked ones are silhouettes.

The poutinerie shows the whole catalogue and how many poutines the user owns. A poutine the user does not own is a black silhouette, with no name and no rarity.

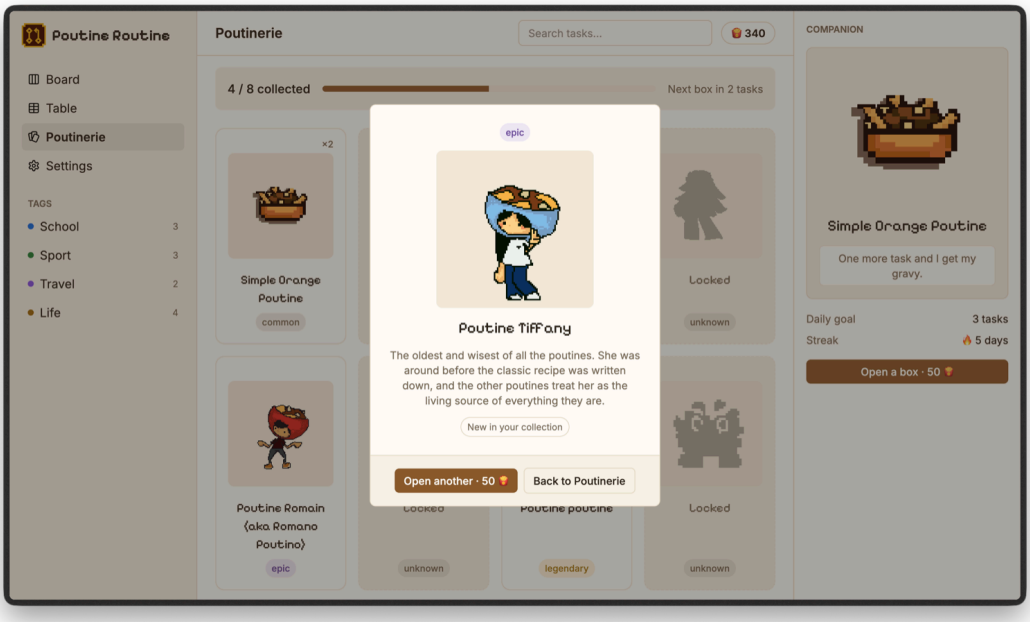


Figure 12: 08 - Opening a box. A random poutine, with its rarity and a short text.

States and settings

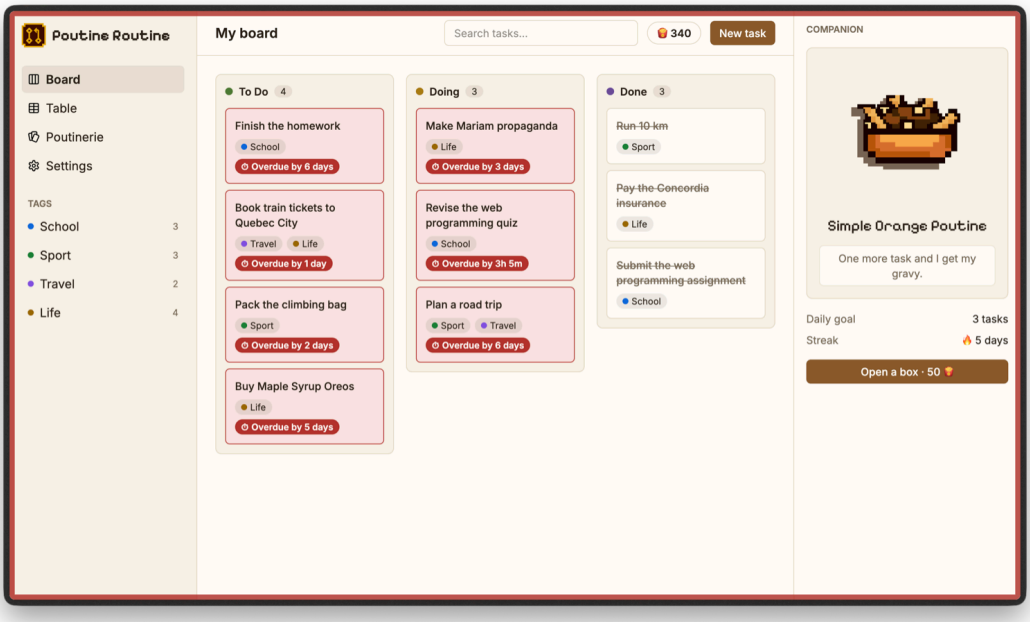


Figure 13: 09 - Overdue. The card turns red and says how late the task is.

When a task passes its due date, its card turns red and the label says “Overdue by”. The edge of the screen pulses red twice. Nothing moves if the system asks for less motion.

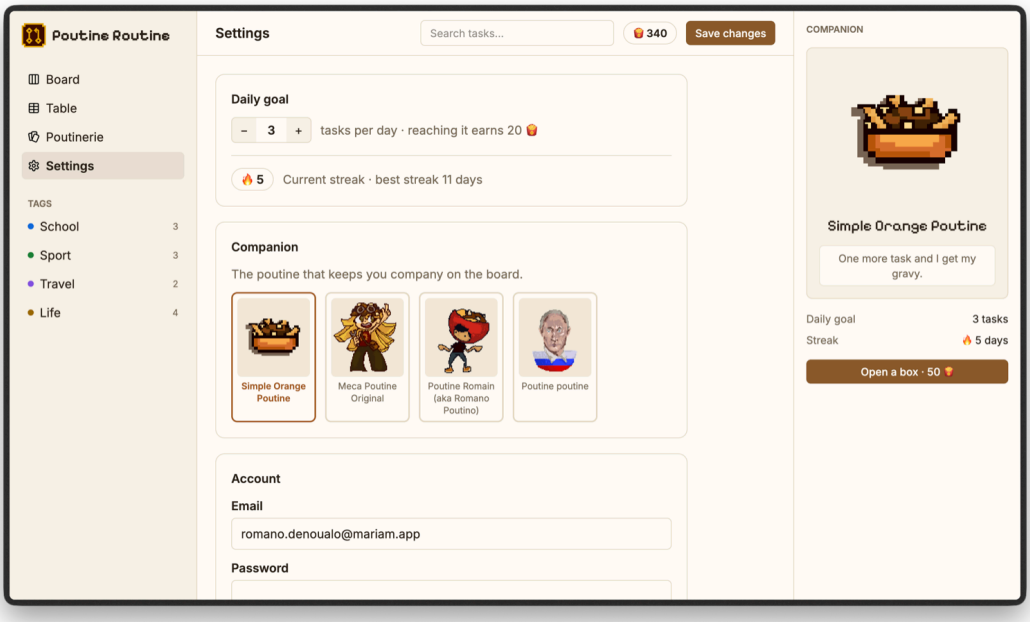


Figure 14: 10 - Settings. Daily goal, streak, companion, account.

Differences from the Milestone 1 mockups

The screens are close to the mockups. Here is a list of the small differences:

- **Left for Milestone 3 because they need the backend.** The “+10 fries” label on done tasks, the “Fed today” bar and the “Earned today” line in the right column, the “Change companion” button, the live “Next box in 2 tasks” message, and the “Time is up” message when a task is late.
- **Simplified on purpose.** The Board and Table pages are two links in the sidebar, not a switch at the top. The task form has a Status field, so the keyboard can move a task. The date and the time are two fields. The columns have no “+” button and the cards have no drag handle.
- **Added.** Sidebar icons, tags that filter the board with a task count, a search box that works, and real pixel art in place of the grey boxes.

Client architecture

Stack as built

Layer	Choice	Note
Framework	Vue 3.5, Options API	`setup()` is used only to expose the store, as in the course
Build tool	Vite 8	Created with `npm create vite@latest`, like the course
Routing	Vue Router 4.6	Six routes
State	Pinia 4	One store, for the tasks
Styling	Tailwind CSS 4.3	With the official Vite plugin. No other CSS file than the theme
HTTP	axios	One shared instance, ready for Milestone 3. No call is made yet
Fonts	Inter and Pixelify Sans	Installed with Fontsource
Icons	Lucide	Four SVG files copied in the project, no icon library
Quality	ESLint 10, Prettier 3.9	Both run on every pull request

The backend (Express, Sequelize, MySQL) is not started. It is the work of Milestone 3.

Folders

```
client/src/
├── assets/       the logo, 26 poutine sprites, 4 sidebar icons
├── components/   10 components
├── mock/         test data: tasks, tags, poutines, user
├── router/       the six routes
├── services/     the shared axios instance
├── stores/       the tasks store
```

```
├── styles/       theme.css, the design tokens
└── views/        one file per screen
```

How the pieces fit

- A **view** is a screen. There is one per route and it always starts with the top bar.
- A **component** is a part of a screen. A modal or a card is a component, not a route.
- The **tasks store** holds the tasks, the tags, the selected tags and the search text. The board, the table, the sidebar and the top bar all read it.
- The **test data** in `src/mock/` fills the store at start. In Milestone 3 the store will fill itself from the API.
- The **router** has one extra setting: login and register are marked as full-screen pages, so the application shows them without the sidebar and the right column.

Route	View
`/`	Board
`/table`	Table
`/poutinerie`	Poutinerie (the collection)
`/settings`	Settings
`/login` and `/register`	Authentication, full screen

Component	Role
`AppSidebar`	Links, tag filter with task counts
`AppTopbar`	Page title, search box, fries balance, main action button
`CompanionRail`	The companion, daily goal, streak and the “Open a box” button
`TaskColumn`	One column of the board. It is also the drop zone
`TaskCard`	One task with its tags and its countdown
`TaskModal`	The form to create or edit a task
`TaskEditor`	Opens the form. The board and the table both use it
`BoxOpeningModal`	The reveal after a box is opened
`PoutineSprite`	The image of one poutine
`MascotSprite`	A random simple poutine, for the login and the empty board

Frontend decisions

One shared store for the tasks

The board, the table, the filter, the search and the task form all work on the same tasks. A Pinia store holds them, so a change shows everywhere at once. The course uses Pinia only for the logged-in user, so this store is an addition of ours. We kept it small:

- **State:** the tasks, the tags, the selected tags and the search text.
- **Getters:** the visible tasks, and the visible tasks of one column.
- **Actions:** add, update, delete and move a task, and turn a tag on or off.

A screen never changes the state by hand: it calls an action. In Milestone 3 the actions will call the API, and the screens will not change.

Design tokens with Tailwind

Colours are tokens, defined once in `src/styles/theme.css` and used as Tailwind classes, for example `bg-bg-muted`, `text-fg-danger` or `border-border-accent`. No component writes a colour by hand.

Drag and drop

We use the events of the browser. A card carries the id of its task when it is picked up. A column accepts the drop, is highlighted while a card is above it, and tells the board which task was dropped. The board then calls the store. A moved card goes to the end of the new column.

Test data that matches the API

The files in `src/mock/` use the database column names, for example `due_at`, `completed_at` and `fries_balance`. A task contains its tags. A poutine with `quantity` 0 is locked. Because the shape is the same, Milestone 3 changes where the data comes from, and not the screens.

Dates: local time and UTC

The server will store dates in UTC, so `due_at` ends with a `Z`. The task form works in local time. The form converts the date in both directions, so a task keeps its hour when it is edited. A date without a time is saved as 23:59 of that day. The countdown updates every minute, and its timer stops when the card leaves the screen.

Pixel-art sprites

Each poutine is one PNG of 96 by 96 pixels, named after its slug. We show it at whole multiples (96 or 192 pixels) with sharp pixels, so the art stays clean. A locked poutine is the same image turned into a black silhouette with CSS.

Quality: lint, format, CI and merge rules

ESLint and Prettier run on every pull request, with the build, in a GitHub Actions workflow. A repository ruleset blocks a merge without a pull request, one

approval and a green check. Merges are squashed, and the pull request title follows Conventional Commits. Each pull request has a “How to verify” list.

We tested the frontend by hand in the browser, with that list. We have no automated tests.

Deployment on GitHub Pages

Every merge into the `main` builds the client and publishes it on GitHub Pages. The site has only the frontend and the test data.

Accessibility

The product has to work for everyone, and a to-do application can be used every day. These points are in the code today:

- **HTML sementic tags:** Every page has one `<main>` and one `<h1>`. Cards, columns, tags and buttons use `<article>`, `<section>`, `<ul>` and `<button>`.
- **Labels on every input:** Each field has a visible label linked to it. Error messages are announced by screen readers.
- **The keyboard reaches the main actions:** The tags of the filter, the title in the table, the cards and the column headers of the table are all buttons.
- **Windows behave:** The task form and the box reveal tell screen readers that they are dialogs, and both close with Escape.
- **Late is not only red:** An overdue task shows the words “Overdue by” and not only a red colour.
- **Less motion:** The red pulse at the edge of the screen runs twice (once per second), and it does not run at all if the system asks for less motion (WCAG 2.3.1).
- **Images:** Decorative images have an empty `alt`, and icon-only buttons have a name.

We did not run a full accessibility audit yet.

Project plan

Planned vs Delivered

This table follows the scope table of Milestone 1.

Feature (priority in Milestone 1)	State	Note
Register, log in, log out, protected session	Screens only	Register and login screens exist. Log out and the session need the backend
Task create, edit, delete, with tags and due date	Done	Title, description, status, due date, priority, tags
Kanban board with drag and drop and a keyboard way	Done	The Status field of the task form is the keyboard way
Tag filter	Done	Several tags at once
Fries on completion	Moved to Milestone 3	The server computes it and pays once per task
Boxes and collection	Collection done, boxes partly	The box shows a random poutine. Paying with fries moves to Milestone 3
Countdown and overdue alert	Done	
Table view	Done	Sort by column
Daily goal and streak	Display only	Counting needs the server
Companion with moods	Not done	The companion shows its sprite and one sentence
Per-poutine effects	Not done	
Dark theme	Not done	The design tokens make it possible
Search	Done	Done earlier than planned
Box animation, statistics, browser notifications	Not done	Optional if we have time

For Milestone 3 we commit to four things that need the backend:

1. fries given when a task is done, once per task
2. buying a box with fries, and drawing the poutine on the server
3. a companion that is chosen in Settings and saved
4. creating, renaming and deleting tags

The other features stay “if time allows”.

Timeline

Period	Sprint	Goal in Milestone 1	State
8 - 21 Sep	0	Concept, scope, data model, mockups, plan	Done. Milestone 1
22 Sep - 4 Oct	1	Vite client, Express server, local MySQL, design tokens, application shell, authentication screens, board with mock data, first sprites	Client, design tokens and first sprites done. Express server and MySQL move to sprint 3
5 - 18 Oct	2	Full frontend on mock data	Done, with the changes above. Milestone 2
19 Oct - 1 Nov	3	Sequelize models, tasks and tags API, authentication, client connected to the API	Next
2 - 15 Nov	4	Economy API, security work, poutine catalogue, tests, documentation	Next. Milestone 3, 16 Nov

Risks

Risk	State	What we do
Authentication is taught late	Open	We start it early, with the course material already online
The frontend comes before the database	Handled	The frontend runs on test data shaped like the database
Pixel art depends on one person	Handled for now	One image per poutine is enough. Moods and animations are optional
Different local MySQL installations	Open	One `.env` file per developer, and a schema that works on MySQL and MariaDB
The game grows and the to-do suffers	Open	The task list comes first. Game rules that need the server wait
Uneven work between members	Open	We update the task table at each milestone
The frontend and the API do not fit	New	The test data uses the database column names
Too little time for security and the game	New	Security and the task list come first, the game waits

How we work

This section does not change since Milestone 1, with one addition: the client is now published on GitHub Pages after each merge into `main`.

Challenges and solutions

Merge conflicts on the router

Problem. Each issue adds its own route in `router/index.js`, so two pull requests often change the same lines. Git then stops with a conflict.

Solution. We kept the router small and chose to learn how to solve these conflicts instead of avoiding them. The rule is simple: keep both sides.

Test data must look like the API

Problem. If the test data does not match the future API, every screen has to be rewritten in Milestone 3.

Solution. The test files use the database column names. We checked them against the database schema. One detail showed up: our poutine texts are longer than the 160 characters planned in the schema, so the column will be made longer when we build the database.

Dates in local time and UTC

Problem. The server stores dates in UTC, but a date field in the browser uses local time. Without a conversion, a task would move by several hours each time it is edited.

Solution. The task form converts the date in both directions. The date and the time are two fields, and a date without a time becomes 23:59.

Highlight that blinked while dragging

Problem. The column under a dragged card lost its highlight each time the mouse crossed a card inside the column.

Solution. The column removes the highlight only if the mouse has really left the column, not when it moves onto one of its children.

Images and Vite

Problem. The sprite of a poutine is chosen by its name at runtime, and Vite could not find the file when the path was built from a name that contained a slash.

Solution. A sprite is named after the slug of its poutine. One small component builds the path from the slug (so the images are found in dev, in the build and on GitHub Pages).

Task distribution

In Milestone 1 we planned one slice per member, from the interface to the database. For the frontend we cut the work by screen, so that everyone could work at the same time. The issues were written in advance, and each member took the ones they wanted. Each change went through a pull request on GitHub.

Milestone 2 - who did what

Contribution	Member	Pull requests
Client setup: Vue project, design tokens, linting, continuous integration, test data	Eliot Cupillard	#4, #14, #15
Favicon	Abigaïl Mialaret	#16
Application shell (sidebar, right column, top bar) and the board with columns and task cards	Eliot Cupillard	#17, #19
Table view with sorting	Romain Denoual	#29
Register screen	Abigaïl Mialaret	#30
Login screen	Tiffany Vongchanh	#31
Tasks store	Eliot Cupillard	#32
Countdown and overdue alert	Eliot Cupillard	#33
Pixel art: 26 poutine sprites	Abigaïl Mialaret	#35
Task form: create, edit, delete	Eliot Cupillard	#36
Settings screen	Tiffany Vongchanh	#37
Poutinerie, the collection screen	Romain Denoual	#38
Drag and drop between columns	Eliot Cupillard	#39
Integration and polish: full-screen login and register, table linked to the store, companion sprite, random mascot	Eliot Cupillard	#40, #41, #42
GitHub Pages deployment	Eliot Cupillard	#43
Box opening	Abigaïl Mialaret	#44
Tag filter and search	Romain Denoual	#45
Page scroll, sidebar icons, “New task” in the table	Eliot Cupillard	#46
This document	All members	

The pull requests are listed in the order of their number. Each number links to the pull request on GitHub. The commits of a pull request are squashed into one, so GitHub shows the author of the pull request.

Tools

Artefact	Tool
Screens in this document	Screenshots of the running application, taken in the browser. The mockups of Milestone 1 stay in Figma
Frontend	Vue 3, Vite, Vue Router, Pinia and Tailwind CSS, with ESLint and Prettier
Pixel-art sprites	Drawn in Pixquare and exported as PNG images
Sidebar icons	Lucide, four SVG files copied into the project (ISC license)
Continuous integration	GitHub Actions: lint, format check and build on every pull request, and the GitHub Pages deployment after each merge
This document	Markdown files in `docs/milestone-2/`, converted to PDF with pandoc and the Typst engine
Source control	Git and GitHub, with issues, pull requests and a branch ruleset
AI assistants	We used AI assistants (Claude, Gemini) at several steps, as learning tools: to understand new notions and the course prerequisites, to learn standards such as accessibility, and to go deeper into topics. Every change went through a pull request, and each member can explain the code they submit